Claims Investigation Copilot

An AI-powered healthcare fraud detection system that uses LangGraph agents, machine learning, and network analysis to automatically identify suspicious billing patterns and generate investigative dossiers.

Overview

This system analyzes medical insurance claims to detect fraud patterns including:

- **Upcoding rings**: Providers billing high-value procedures at rates far exceeding peers
- **Phantom billing**: Billing for services never rendered
- **Doctor shopping**: Patients seeking controlled substances from multiple providers
- **Referral kickback schemes**: Suspicious referral pattern shifts indicating illegal arrangements

Features

- **50K synthetic claims** with realistic fraud patterns (95% normal, 5% fraud)
- **Anomaly detection** using Isolation Forest ML
- **Network graph analysis** with NetworkX (providers, members, facilities)
- **Semantic billing rules search** using FAISS vector index
- **Interactive visualizations** with PyVis
- **3 LangGraph AI agents** (orchestrator, investigation, dossier)
- **Hero demo case**: $2.3M upcoding ring with 4 providers and 12 patients

Architecture

```
Stack:
├── LangGraph + Claude (Bedrock) - Multi-agent orchestration
├── Pandas - Data processing (50K claims, 500 providers, 5K members)
├── Scikit-learn - Anomaly detection (Isolation Forest)
├── NetworkX - Graph analysis (fraud rings, connections)
├── FAISS - Vector search over CMS/OIG billing rules
└── PyVis - Interactive network visualization
```

3 AI Agents

1. **Orchestrator** (Lead Investigator) - Delegates tasks, no tools
2. **Investigation Agent** (Detective) - 7 tools to scan/profile/analyze
3. **Dossier Agent** (Case Writer) - 5 tools to assess/compile evidence

Installation

Prerequisites

- Python 3.12+
- pip

Setup

1. **Clone/navigate to the project:**

```
cd claims-copilot
```

2. **Create virtual environment (recommended):**

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1  # Windows
# or
source .venv/bin/activate  # macOS/Linux
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

Quick Start

1. Configure AWS Bedrock

Create `.env` file (copy from `.env.example`):

```
AWS_PROFILE=your-profile-name
AWS_REGION=us-east-1
```

Ensure your AWS CLI is configured with Bedrock access.

2. Run the Data Foundation Demo

```
python main.py
```

This will:

1. Generate 50K synthetic claims with fraud cases embedded
2. Compute provider/member features (13 metrics per provider)
3. Run anomaly detection (Isolation Forest)
4. Build network graph (providers → members → facilities)
5. Initialize billing rules index (FAISS)
6. **Run discovery demo** showing how the hero case is found

3. Run the Full AI Agent Investigation

```
python run_agents.py
```

This will:

1. Initialize all data (same as main.py)
2. Launch 3 AI agents (Orchestrator, Investigation, Dossier)
3. Automatically investigate the hero case (P-4482)
4. Generate a complete investigation dossier in `investigation_dossier.md`

The agents use AWS Bedrock Claude 3.5 Sonnet to autonomously:

- Scan for anomalies
- Profile suspicious providers
- Find fraud rings and connections
- Search billing rules violations
- Assess evidence sufficiency
- Compile comprehensive investigation reports

Expected Output

```
============================================================
CLAIMS INVESTIGATION COPILOT - Data Initialization
============================================================

[1/5] Generating synthetic data...
Providers: 500
Members: 5000
Facilities: 100
Claims: 50000+
Total billed: $XXX,XXX,XXX

[2/5] Computing features...
Provider features: 500 providers
Member features: 5000 members

[3/5] Running anomaly detection...
Total entities scored: 5500
Entities with anomaly score > 0.8: ~25
Hero case (P-4482) anomaly score: 0.92 ✓

[4/5] Building network graph...
Graph built: XXXX nodes, YYYY edges

[5/5] Initializing billing rules index...
Loaded 12 rules and 3 patterns

============================================================
DEMO: Discovering the Hero Case
============================================================

[Step 1] Scanning for high-anomaly providers...
Found X providers with anomaly score > 0.7:
  • P-4482: score=0.92, billed=$2,300,000

[Step 2] Profiling P-4482...
  Specialty: Orthopedic Surgery
  CPT Concentration: 89% (peer avg: 23%)
  Referral Concentration: 78%
  Top anomaly features:
    - top_cpt_concentration: 0.89 (z=5.2)
    - referral_concentration: 0.78 (z=4.8)
    - avg_billed_per_claim: 45,000 (z=3.9)

[Step 3] Finding connections...
  Connected entities: 15+
  High-anomaly connected providers:
    • P-1190 (anomaly=0.65) - referring provider
    • P-3387 (anomaly=0.58) - referring provider
    • P-5521 (anomaly=0.61) - referring provider

[Step 4] Finding fraud rings...
Found 1 potential ring
  Ring 1:
    Entities: 16 (4 providers, 12 members)
    Total billed: $2,300,000
    Providers: ['P-4482', 'P-1190', 'P-3387', 'P-5521']

[Step 5] Analyzing referral history...
  Shows 78% referral concentration shift ~8 months ago

[Step 6] Searching relevant billing rules...
  • CMS-27447-002: TKA Billing Frequency Standards
  • OIG-UPCODING-001: Upcoding - General Definition
  • OIG-KICKBACK-001: Anti-Kickback Referral Patterns
```

Project Structure

```
claims-copilot/
├── README.md                      # This file
├── requirements.txt               # Python dependencies
├── .env.example                   # AWS config template
├── main.py                        # Data foundation demo
├── run_agents.py                  # AI agent investigation runner ⭐
│
├── data/                          # Data layer
│   ├── __init__.py
│   ├── generate_synthetic.py      # Synthetic claims generation
│   ├── features.py                # Feature computation (13 metrics)
│   ├── anomaly.py                 # Isolation Forest anomaly detection
│   └── graph.py                   # NetworkX graph builder
│
├── agents/                        # LangGraph AI agents ✅ IMPLEMENTED
│   ├── __init__.py
│   ├── prompts.py                 # Agent system prompts
│   ├── tools_investigation.py     # 7 investigation tools
│   ├── tools_dossier.py          # 5 dossier tools
│   ├── nodes.py                   # Agent node implementations (Bedrock)
│   └── graph.py                   # LangGraph state machine
│
├── billing_rules/                 # Regulatory search
│   ├── __init__.py
│   ├── rules.json                 # CMS/OIG guidelines corpus
│   └── index.py                   # FAISS semantic search
│
└── viz/                           # Visualization
    ├── __init__.py
    └── ring_viz.py                # PyVis interactive graphs
```

Hero Case Details

The demo focuses on discovering this embedded fraud pattern:

Pattern: Upcoding Ring with Referral Kickback
Primary: Provider P-4482 (Orthopedic Surgery)
Referring Providers: P-1190, P-3387, P-5521
Patients: 12 shared members
Claims: 47 suspicious claims
Total Billed: $2,300,000
Recoverable: ~$252,600

Red Flags:

- CPT 27447 (knee replacement) at 89% vs peer avg 23%
- 3 referrers shifted 78% of referrals to P-4482 ~8 months ago
- Anomaly score: 0.92 (very high)
- All operations at same facility (F-9901)

Data Schema

Source DataFrames

claims_df (50K rows):

- claim_id, member_id, provider_id, referring_provider_id
- facility_id, cpt_code, icd_code
- billed_amount, paid_amount, service_date
- place_of_service, claim_type

providers_df (500 rows):

- provider_id, specialty, region, peer_group

members_df (5K rows):

- member_id, age, gender, region

Derived Data

provider_features_df - 13 metrics per provider:

- claims_per_month, unique_patients_per_month
- pct_high_complexity, avg_billed_per_claim
- top_cpt_concentration, referral_concentration
- duplicate_claim_rate, etc.

anomaly_scores_df - Isolation Forest scores:

- entity_id, entity_type, anomaly_score (0-1)
- top_features (contributing factors)
- total_billed

graph - NetworkX DiGraph:

- Nodes: providers, members, facilities
- Edges: BILLED_FOR, REFERRED_TO, OPERATES_AT

Next Steps

For Frontend Integration

The data layer is complete and ready. When building the FE:

1. **API Endpoints Needed**:
   - `GET /scan?threshold=0.7` - High anomaly entities
   - `GET /provider/:id/profile` - Full provider profile
   - `GET /provider/:id/connections` - Network analysis
   - `GET /rings?min_anomaly=0.5` - Fraud ring detection
   - `POST /visualize/ring` - Generate PyVis HTML
2. **Data Context** - Use `DataContext` class in `main.py`:

```
from main import initialize_data
ctx = initialize_data()

# Access methods
ctx.get_high_anomaly_providers(threshold=0.7)
ctx.get_provider_profile("P-4482")
ctx.find_provider_connections("P-4482", depth=2)
ctx.find_fraud_rings(min_anomaly=0.5)
```

3. **Visualization** - PyVis outputs are HTML files:

```
from viz import visualize_ring
visualize_ring(ring_data, "output.html")
# Embed in iframe or serve static file
```

Agent Implementation Details

The agent system is **fully implemented** with AWS Bedrock Claude 3.5 Sonnet.

3 Agents:

1. **Orchestrator** - Routes between investigation and dossier phases
2. **Investigation Agent** - 7 tools (scan_new_claims, profile_entity, compare_to_peers, get_claim_details, find_connections, find_ring, get_referral_history)
3. **Dossier Agent** - 5 tools (assess_evidence, search_billing_rules, find_similar_cases, estimate_recovery, compile_dossier)

Workflow:

```
START → Orchestrator → Investigation Agent → Orchestrator
                            ↓                      ↓
                      Dossier Agent ←──────────────┘
                            ↓
                      investigation_dossier.md
```

To Run: `python run_agents.py` (requires AWS Bedrock access)

Testing

Run individual modules:

```
# Test data generation only
python data/generate_synthetic.py

# Test feature computation
python data/features.py

# Test anomaly detection
python data/anomaly.py

# Test graph building
python data/graph.py

# Test billing rules search
python billing_rules/index.py

# Test visualization
python viz/ring_viz.py
```

Technologies

- **LangGraph** - Multi-agent orchestration
- **Pandas** - Data manipulation
- **NumPy** - Numerical operations
- **Scikit-learn** - Machine learning (Isolation Forest)
- **NetworkX** - Graph analysis
- **FAISS** - Vector similarity search
- **PyVis** - Network visualization
- **AWS Bedrock** - Claude LLM (planned)

License

Hackathon project - Internal use

Authors

Built for healthcare fraud detection hackathon 2026